
Classification Models for Subreddit Identification

Cher Li

Department of Computer Engineering
McGill University
cher.li@mail.mcgill.ca

Nara Yun

Department of Software Engineering
McGill University
nara.yun@mail.mcgill.ca

Karen Cherry Chen Lai

Department of Computer Engineering
McGill University
karen.chenlai@mail.mcgill.ca

Abstract

This project investigates the performance of various classifier models, both implemented from scratch and from scikit-learn, on a benchmark test dataset gathered from Reddit posts. The models will classify text data into one of four subreddits after supervised learning on a separate dataset. The performance of all models is ranked using k-fold cross validation from scikit-learn to explore the accuracy of the models. The best model is then used to predict the subreddit of the test dataset, and the resulting csv file will be submitted to the Kaggle competition, ECSE 551 W25 MP2. After model experimentation, the best standalone model is logistic regression on tfidf vectorizer data, while the best overall model is the voting classifier on count vectorizer data.

1 Introduction

Machine Learning (ML) is applied to numerous fields of study, from healthcare to communication. This study focuses on textual data instead of numerical data and an exploration of the Bernoulli Naïve Bayes model. This project also involves using scikit-learn to develop classification models for analyzing the textual data. The models take a csv file of Reddit posts and predict the labels of which subreddit community the posts belong to. Numerous models and data preprocessing changes are conducted to reveal the best accuracy: result tables below show the different accuracies based on different datasets and changes such as removing the negation from the list of stop words.

Logistic regression trained on tfidf vectorizer data results in an accuracy of 0.7, while a voting classifier (estimators of naive bayes, logistic regression, decision tree, random forest, and mlp classifier) with negation removed results in 0.72. Also, this logistic regression model takes 25.76 seconds to complete, while the voting classifier takes up to 482.10 seconds, showing the tradeoff between accuracy and efficiency.

2 Dataset

The dataset consists of Reddit posts from four subreddits: Ottawa, Geneva, Canberra, and Boston. It includes train.csv with 1400 labeled posts and test.csv with 600 unlabeled posts for Kaggle submission. To enhance classification accuracy, we performed exploratory data analysis (EDA) and applied various preprocessing techniques.

2.1 Exploratory Data Analysis

We conducted exploratory data analysis (EDA) to assess dataset structure, word distributions, and linguistic composition.

Word frequency analysis confirmed dataset stability, with a strong overlap in frequent words between train.csv and test.csv. The frequent occurrence of subreddit-related location names suggested their importance for classification [1]. Word cloud visualizations reinforced dataset consistency, highlighting common terms such as *people*, *time*, *work*, *need*, *good*, and validating our choice of small α values in Naïve Bayes to prevent overfitting.

Sentence length analysis revealed a right-skewed distribution, with most posts between 20 and 50 words. Given that sentence length influences text classification performance [2], the observed distribution supported using binary feature representation, particularly for Bernoulli Naïve Bayes, which performs well on shorter texts.

Language detection identified both English and French posts, prompting the inclusion of bilingual stopword lists. Removing French-accented words lowered accuracy, confirming that retaining multilingual text improved model performance. These insights guided preprocessing decisions, ensuring optimal feature selection for classification.

2.2 Text Preprocessing

Text was normalized by lowercasing, removing URLs, punctuation, and numbers. Tokenization and stopword removal were applied for English and French. Expanding contractions (*don't* → *do not*) improved text consistency. Lemmatization was used for English, while stemming was applied for French [3].

A custom tokenizer integrated these transformations and handled part-of-speech tagging. Removing French-accented words lowered accuracy, confirming the necessity of retaining bilingual text.

2.3 Feature Engineering

We experimented with TF-IDF vectorization to capture term importance and CountVectorizer with binary encoding for Bernoulli Naïve Bayes [4]. To improve generalization, we combined both into a hybrid representation, merging binary features with unique TF-IDF terms.

2.4 Final Data Representations

Three datasets were generated: one using TF-IDF, one using binary CountVectorizer, and a final hybrid representation combining both. These structured representations ensured robust model performance.

3 Bernoulli Naïve Bayes

The Bernoulli Naïve Bayes model is implemented as a class from scratch [5]. This model assumes that features are binary indicators, where each word's presence or absence in a document contributes to classification. The class maintains three key components: `class_priors`, which stores $P(y = c)$ for prior probabilities; `feature_probs`, which stores the conditional probability of each binary feature being equal to 1; and `classes`, which contains the unique class labels.

The fit function trains the classifier using a binary feature matrix and class labels. After extracting all unique class labels, the prior probability for each class is calculated as:

$$P(y) = \frac{\sum_{i=1}^n \mathbb{1}(y_i = c)}{n}$$

where $\mathbb{1}(y_i = c)$ is an indicator function that returns 1 if $y_i = c$, and n is the total number of samples.

For each class c , the conditional probability of each word appearing given the class is estimated using the Maximum Likelihood Estimation with Laplace smoothing:

$$P(x_j = 1|y = c) = \frac{\sum_{i=1}^n x_{ij} + \alpha}{\sum_{i=1}^n 1 + 2\alpha}$$

where x_{ij} is the presence (1) or absence (0) of feature j in sample i , and α is the smoothing parameter. The predict function uses the learned parameters to classify new samples. For each input feature vector x , it computes the posterior probability:

$$P(y|x) \propto P(y) \prod_{j=1}^m P(x_j|y)$$

where m is the total number of features. The function returns the class y that maximizes this probability.

Since probabilities can become extremely small, calculations are performed in logarithmic space to prevent underflow:

$$\log P(y|x) = \log P(y) + \sum_{j=1}^m x_j \log P(x_j|y) + (1 - x_j) \log(1 - P(x_j|y))$$

The evaluate function computes standard classification metrics: accuracy, precision, recall, and F1 score. These are defined as:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where TP , FP , and FN represent true positives, false positives, and false negatives, respectively.

This implementation ensures an interpretable yet effective classification model for text data.

4 Proposed Approach

Numerous other classifier algorithms are tested using the train and test data, from logistic regression to MLPClassifier [6-7]. For each model, each of the three dataset is applied, resulting in the comparison of different combinatory accuracies. For KNN algorithm, k is varied as odd numbers between 1 and 30, but due to the subpar accuracy, the algorithm is not included in the results table. Finally, many trials and ways of data preprocessing are conducted on the voting classifier for both hard and soft voting.

5 Results

Table 1: Accuracy Results

Algorithm	Count Vectorizer	Tfidf Vectorizer	Mixed
Logistic Regression	0.66	0.70	0.63
Decision Tree	0.57	0.57	0.54
Random Forest	0.66	0.67	0.63
SVC	0.61	0.69	0.58
BernoulliNB	0.64	0.62	0.64
MLPClassifier	0.68	0.67	0.64

Table 2: Accuracy Results for Lemmatization only

Algorithm	Count Vectorizer	Tfidf Vectorizer	Mixed
Logistic Regression	0.66	0.70	0.65
Decision Tree	0.56	0.57	0.57
Random Forest	0.66	0.65	0.66
SVC	0.61	0.69	0.60
BernoulliNB	0.64	0.62	0.66
MLPClassifier	0.68	0.68	0.66

Table 3: Accuracy Results for Stemming only

Algorithm	Count Vectorizer	Tfidf Vectorizer	Mixed
Logistic Regression	0.66	0.70	0.64
Decision Tree	0.56	0.56	0.57
Random Forest	0.66	0.66	0.64
SVC	0.61	0.69	0.58
BernoulliNB	0.64	0.62	0.65
MLPClassifier	0.68	0.67	0.65

Finally, the majority of the testing revolves around the voting classifier [8]. Combinations of soft or hard voting, different datasets, and outside changes reveal the best model accuracy.

Table 4: Accuracy Results for Soft Voting Classifier

Changes	Mixed	Tfidf	Count
Only lemmatization	0.69	0.71	0.70
Only stemming	0.67	0.70	0.70
Removing negation	0.67	0.70	0.71

Table 5: Accuracy Results for Hard Voting Classifier

Changes	Mixed	Tfidf	Count
Only lemmatization	0.70	0.71	0.70
Only stemming	0.67	0.71	0.69
Removing negation	0.67	0.71	0.72

Different numbers and types of models for the voting classifier are also tested for the mixed dataset. As the hard voting tend to have a lower accuracy, the results table below will only be showing the soft voting. Please refer to the code in appendix B for what each individual model entails.

Table 6: Accuracy Results for Soft Voting Combinations

Number	D	E	F	G	H
1	0.6688	0.6816	0.6709	0.6660	0.6809
2	0.6331	0.6688	0.6688	0.6909	0.6703
3	0.6510	0.6759	0.6595	0.6702	*
4	0.6617	0.6781	0.6645	*	*
5	0.6688	0.6595	0.6909	*	*
6	*	0.6388	0.6652	*	*
7	*	0.6874	0.6859	*	*
8	*	0.6702	0.6638	*	*
9	*	0.6781	*	*	*
10	*	0.6631	*	*	*
11	*	0.6638	*	*	*
12	*	0.6631	*	*	*
13	*	0.6888	*	*	*

6 Discussion and Conclusion

For single algorithms, Logistic Regression with TF-IDF achieved the highest accuracy (0.70). Bernoulli Naive Bayes performed well (0.64) with binary Count Vectorization, while Decision Trees and Random Forests underperformed (0.54-0.67), likely due to overfitting.

Model selection depends on the task. Logistic Regression is efficient and interpretable, Naive Bayes suits short text, and neural networks showed no significant improvement, suggesting deep learning is unnecessary for small datasets.

Voting classifier improves accuracy, reaching 0.71 with TF-IDF and 0.72 with Count Vectorizer and data preprocessing adjustments, demonstrating ensemble learning’s benefits.

In the end, the best model is utilized to predict the labels for the test data. The resulting csv file is submitted online for the Kaggle competition with a public accuracy of 0.7222.

The best standalone model was Logistic Regression, while the voting classifier provided the most robust results. Future work could explore boosting, bagging, contextual embeddings like BERT, and increased experiments regarding validation splits / regularization / hyper-parameters, to further increase accuracy for the benefit of classification.

Statement of Contributions

This project is a team effort, and each team member has contributed an equal amount of time to reach the current accuracy results.

Nara Yun: Coded the three different datasets and Bernoulli Naive Bayes

Cher Li: Imported data, stopwords preprocessing, tested other classifiers, submitted results on kaggle

Karen Cherry Chen Lai: Data preprocessing like lemmatization, focused on voting classifier and model combinations

References

- [1] Y. Ko and J. Seo, “Text classification based on the bias of word frequency over categories,” *Information Processing & Management*, vol. 42, no. 1, pp. 240–247, 2006. [Online]. Available: https://www.researchgate.net/publication/221173682_Text_Classification_based_on_the_Bias_of_Word_Frequency_over_Categories
- [2] J. Liu, N. Yang, and H. Liu, “Distribution of sentence length of English complex sentences,” *Glottometrics*, vol. 36, pp. 1–17, 2016. [Online]. Available: https://www.researchgate.net/publication/387227966_Distribution_of_sentence_length_of_English_complex_sentences


```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import time
import sys
import nltk
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
from sklearn.preprocessing import Normalizer
from sklearn.feature_extraction import text
from nltk import word_tokenize
from sklearn.naive_bayes import BernoulliNB
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from sklearn.neural_network import MLPClassifier
from nltk.stem import WordNetLemmatizer
from nltk.stem import PorterStemmer
from nltk.corpus import wordnet
from nltk import pos_tag, word_tokenize
from nltk.corpus import stopwords
import nltk
import re
from sklearn.linear_model import LogisticRegression,SGDClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.model_selection import KFold
from sklearn.tree import DecisionTreeClassifier
from google.colab import drive
drive.mount('/content/gdrive')

```

Data Import and Processing

```

from google.colab import drive
drive.mount('/content/gdrive')
import os

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')
nltk.download('stopwords')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
my_stop_words = list(text.ENGLISH_STOP_WORDS)
french_stop_words = set(stopwords.words('french'))

folder_path = "/content/drive/MyDrive/ECSE551"
print("Folder exists:", os.path.exists(folder_path))
print("Files in ECSE551:", os.listdir(folder_path) if os.path.exists(folder_path) else "Folder not found")

train = pd.read_csv('/content/drive/MyDrive/ECSE551/train.csv', header=None, encoding='latin-1')
train[1].value_counts()
print(train[1].value_counts())
train_data = train[0].tolist()
print(train_data)

test = pd.read_csv('/content/drive/MyDrive/ECSE551/test.csv', encoding='latin-1')
test.columns = ['id', 'text']
test['text'] = test['text'].apply(lambda x: re.sub(r'[\W\s]', ' ', str(x)))
test_data = test['text'].tolist()

```

Enhance Data preprocessing

```

import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer, SnowballStemmer

# Download required NLTK resources
nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("omw-1.4")

# Initialize English Lemmatizer & French Stemmer
lemmatizer = WordNetLemmatizer()
stemmer_fr = SnowballStemmer("french")

# altering stop words list
custom_english = ["i", "me", "my", "myself", "we", "our", "ours", "ourselves",
"you", "your", "yours", "yourself", "yourselves", "he", "him",
"his", "himself", "she", "her", "hers", "herself", "it", "its",
"itself", "they", "them", "their", "theirs", "themselves",
"what", "which", "who", "whom", "this", "that", "these", "those",
"am", "is", "are", "was", "were", "be", "been", "being",
"have", "has", "had", "having", "do", "does", "did", "doing",
"a", "an", "the", "and", "but", "if", "or", "because", "as",
"until", "while", "of", "at", "by", "for", "with", "about",
"against", "between", "into", "through", "during", "before",
"after", "above", "below", "to", "from", "up", "down", "in",
"out", "on", "off", "over", "under", "again", "further",
"then", "once", "here", "there", "when", "where", "why",
"how", "all", "any", "both", "each", "few", "more", "most",
"other", "some", "such", "only", "own",
"same", "so", "than", "too", "very", "s", "t", "can", "will",
"just", "don", "should", "now"]
# removed "no", "nor", "not"

```

```

# Define manual contractions dictionary (for English)
contractions_dict = {
    "i'm": "i am", "you're": "you are", "he's": "he is", "she's": "she is", "it's": "it is",
    "we're": "we are", "they're": "they are", "i've": "i have", "you've": "you have",
    "we've": "we have", "they've": "they have", "isn't": "is not", "aren't": "are not",
    "wasn't": "was not", "weren't": "were not", "haven't": "have not", "hasn't": "has not",
    "hadn't": "had not", "won't": "will not", "wouldn't": "would not", "don't": "do not",
    "doesn't": "does not", "didn't": "did not", "can't": "cannot", "couldn't": "could not",
    "shouldn't": "should not", "mightn't": "might not", "mustn't": "must not"
}

def expand_contractions(text):
    """Replace contractions in text using predefined dictionary (English only)."""
    words = text.split()
    expanded_words = [contractions_dict[word] if word in contractions_dict else word for word in words]
    return " ".join(expanded_words)

# Language detection (simple heuristic using word frequency)
def detect_language(text):
    """Detects language based on common French & English stopwords."""
    french_stopwords = set(stopwords.words('french'))
    english_stopwords = set(stopwords.words('english'))

    words = text.split()
    french_count = sum(1 for word in words if word in french_stopwords)
    english_count = sum(1 for word in words if word in english_stopwords)

    return "fr" if french_count > english_count else "en"

```

```
# Function for cleaning text (Handles English & French)
def clean_text(text):
    if not isinstance(text, str): # Ensure input is a string
        return ""

    # Convert to lowercase
    text = text.lower()

    # Expand contractions (Only for English)
    text = expand_contractions(text)

    # Remove URLs
    text = re.sub("http[s]?www.\S+", "", text)

    # Remove punctuation
    text = re.sub("[^\w\s]", "", text)

    # Remove numbers
    text = re.sub("\d+", "", text)

    # Remove extra spaces
    text = re.sub("\s+", " ", text).strip()

    # Detect language
    lang = detect_language(text)

    # Tokenization & Lemmatization/Stemming
    words = text.split()
    if lang == "en":
        words = [lemmatizer.lemmatize(word) for word in words if word not in stopwords.words("english")]
    elif lang == "fr":
        words = [lemmatizer.lemmatize(word) for word in words if word not in custom_english]
    else:
        words = [stemmer.stem(word) for word in words if word not in stopwords.words("french")]

    return " ".join(words)

# Apply preprocessing using *list comprehensions* instead of *.apply()*
train_data = [clean_text(text) for text in train_data] # List of cleaned training texts
test_data = [clean_text(text) for text in test_data] # List of cleaned test texts
```

Datasets

```
] TF_IDF_vectorizer = TfidfVectorizer(
    lowercase=True,
    stop_words='english',
    max_features=3000,
    max_df=0.7,
    ngram_range=(1, 2)
)

# Fit TF-IDF and transform data
train_tfidf = TF_IDF_vectorizer.fit_transform(train_data)
test_tfidf = TF_IDF_vectorizer.transform(test_data)
selected_features = set(TF_IDF_vectorizer.get_feature_names_out())

# Filter shape of the data
print("TF-IDF Vectorized Data Shape:", train_tfidf.shape, test_tfidf.shape)

train_tfidf = train_tfidf.toarray()
test_tfidf = test_tfidf.toarray()

# Store tfidf dataset
pd.DataFrame(train_tfidf).to_csv('train_tfidf.csv', index=False, header=False)
pd.DataFrame(test_tfidf).to_csv('test_tfidf.csv', index=False, header=False)
```

```
def map_pos_tag(word):
    """
    Convert a POS tag to the format required by WordNet lemmatizer.
    """
    pos = nltk.pos_tag(word)[0][1].upper()
    pos_mapping = {"J": "a", "M": "m", "V": "v", "N": "n"}
    return pos_mapping.get(pos, "n") # default to noun if not found

class CustomTokenizer:
    def __init__(self, feature_set):
        self.lemmatizer = WordNetLemmatizer()
        self.stemmer = PorterStemmer()
        self.feature_set = feature_set # Store the feature set for filtering

    def __call__(self, text):
        """
        Tokenizes, lemmatizes, and stems the input text while filtering words.
        """
        words = word_tokenize(text)
        lemmatized = [self.lemmatizer.lemmatize(word, pos=map_pos_tag(word)) for word in words if word.isalpha()]
        stemmed = [self.stemmer.stem(word) for word in lemmatized]
        return [word for word in stemmed if word in self.feature_set]
```

```
# Apply binary vectorization using a customized tokenizer
binary_vectorizer = CountVectorizer(
    tokenizer=CustomTokenizer(selected_features),
    lowercase=True, # Ensure text is converted to lowercase
    binary=True # Binary encoding for BernoulliNB compatibility
)
X_train_bin = binary_vectorizer.fit_transform(train_data)
X_test_bin = binary_vectorizer.transform(test_data)

# Display the shape of the binary-transformed training data
X_train_bin.shape
```

```
Tf_idf_vectorizer = TfidfVectorizer(
    tokenizer=CustomTokenizer(selected_features),
    max_features=3000
)

# Learn vocabulary from training data and transform it into a TF-IDF matrix
X_train_tfidf_matrix = Tf_idf_vectorizer.fit_transform(train_data)

# Apply the same transformation to the test set
X_test_tfidf_matrix = Tf_idf_vectorizer.transform(test_data)

# Display the dimensions of the transformed training dataset
X_train_tfidf_matrix.shape
```

```

from scipy.sparse import hstack

# Extract vocabulary lists from both feature extraction techniques
binary_vocab = set(binary_vectorizer.get_feature_names_out())
tfidf_vocab = set(tfidf_vectorizer.get_feature_names_out())

# Identify terms that exist in both feature sets
shared_terms = binary_vocab.intersection(tfidf_vocab)

# Locate indices of overlapping features within the TF-IDF matrix
excluded_tfidf_indices = [
    idx for idx, term in enumerate(tfidf_vectorizer.get_feature_names_out()) if term in shared_terms
]

# Retain only unique features in the TF-IDF representation
X_train_tfidf_unique = X_train_tfidf_matrix[:, [col for col in range(X_train_tfidf_matrix.shape[1]) if col not in excluded_tfidf_indices]]
X_test_tfidf_unique = X_test_tfidf_matrix[:, [col for col in range(X_test_tfidf_matrix.shape[1]) if col not in excluded_tfidf_indices]]

# Merge binary and filtered TF-IDF matrices
X_train_combined = hstack([X_train_bin, X_train_tfidf_unique])
X_test_combined = hstack([X_test_bin, X_test_tfidf_unique])

# Convert sparse matrices to dense format
X_train_combined_dense = X_train_combined.toarray()
X_test_combined_dense = X_test_combined.toarray()

# Save the final processed datasets
pd.DataFrame(X_train_combined_dense).to_csv('processed_X_train.csv', index=False, header=False)
pd.DataFrame(X_test_combined_dense).to_csv('processed_X_test.csv', index=False, header=False)

# Confirm processing completion
print("Combined feature matrices stored as CSV files: processed_X_train, processed_X_test")

```

```

train_df = pd.DataFrame(train)
test_df = pd.DataFrame(test)
word_vectorizer = CountVectorizer(stop_words='english')

# Apply vectorization on the first column of the dataset (text data)
train_word_counts = word_vectorizer.fit_transform(train_df.iloc[:, 0])
test_word_counts = word_vectorizer.transform(test_df.iloc[:, 0])

# Convert sparse matrices to DataFrames for better handling
train_word_df = pd.DataFrame(train_word_counts.toarray(), columns=word_vectorizer.get_feature_names_out())
test_word_df = pd.DataFrame(test_word_counts.toarray(), columns=word_vectorizer.get_feature_names_out())

# Convert DataFrames to NumPy arrays for model input
X_train_basic = train_word_df.to_numpy()
X_test_basic = test_word_df.to_numpy()
pd.DataFrame(X_train_basic).to_csv('basic_train_features.csv', index=False, header=False)
pd.DataFrame(X_test_basic).to_csv('basic_test_features.csv', index=False, header=False)

y_labels = train_df.iloc[:, 1]

pd.DataFrame(y_labels).to_csv('train_labels.csv', header=None, index=None)

```

Bernoulli Naïve Bayes

```

class BernoulliNaiveBayes:
    def __init__(self):
        self.class_priors = {} # Stores P(y=c)
        self.feature_probs = {} # Stores P(xj=1 | y=c)
        self.classes = None

    def fit(self, X, y):
        """
        Train the Bernoulli Naive Bayes classifier.
        :param X: Binary feature matrix (values 0 or 1)
        :param y: Class labels
        """
        self.classes = np.unique(y)
        total_samples, total_features = X.shape

        # Compute class prior probabilities
        for label in self.classes:
            X_c = X[y == label]
            self.class_priors[label] = len(X_c) / total_samples # P(y=label)

        # Compute feature probabilities with Laplace smoothing
        self.feature_probs[label] = (np.sum(X_c, axis=0) + 1) / (len(X_c) + 2)

```

```

def predict(self, X):
    """
    Predict class labels for input samples.
    :param X: Binary feature matrix
    :return: Predicted class labels
    """
    return np.array([self._predict_instance(sample) for sample in X])

def _predict_instance(self, sample):
    """
    Predict the class for a single instance.
    :param sample: A binary feature vector
    :return: Predicted class label
    """
    log_probs = {}

    for label in self.classes:
        log_prob = np.log(self.class_priors[label])
        feature_probs = self.feature_probs[label]
        log_prob += np.sum(sample * np.log(feature_probs) + (1 - sample) * np.log(1 - feature_probs))
        log_probs[label] = log_prob

    return max(log_probs, key=log_probs.get)

```

```

def evaluate(self, y_pred, y_true):
    """
    Compute evaluation metrics (accuracy, precision, recall, and F1 score).
    :param y_pred: Predicted labels
    :param y_true: True labels
    :return: Dictionary containing the evaluation metrics
    """
    y_pred, y_true = np.array(y_pred), np.array(y_true)

    metrics = {
        'accuracy': np.mean(y_pred == y_true),
        'precision_macro': 0,
        'recall_macro': 0,
        'f1_macro': 0,
        'precision_weighted': 0,
        'recall_weighted': 0,
        'f1_weighted': 0
    }

    class_precisions, class_recalls, class_f1s = [], [], []
    class_counts = [np.sum(y_true == label) for label in self.classes]

    for label in self.classes:
        tp = np.sum((y_pred == label) & (y_true == label))
        fp = np.sum((y_pred == label) & (y_true != label))
        fn = np.sum((y_pred != label) & (y_true == label))

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0
        f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0

        class_precisions.append(precision)
        class_recalls.append(recall)
        class_f1s.append(f1)

    metrics['precision_macro'] = np.mean(class_precisions)
    metrics['recall_macro'] = np.mean(class_recalls)
    metrics['f1_macro'] = np.mean(class_f1s)

    total_samples = len(y_true)
    metrics['precision_weighted'] = np.sum(np.array(class_precisions) * class_counts) / total_samples
    metrics['recall_weighted'] = np.sum(np.array(class_recalls) * class_counts) / total_samples
    metrics['f1_weighted'] = np.sum(np.array(class_f1s) * class_counts) / total_samples

    return metrics

```

```

from sklearn.model_selection import KFold
kf = KFold(n_splits=10, shuffle=True, random_state=40)
# Extract labels from train dataset
y = np.array(train[1]) # Ensure it's an array and aligns with train_tfidf
print(y)

## Train the training data after tf-idf
model = BernoulliNaiveBayes()
accuracies = []
fold = 1
start_time = time.time()
for train_index, test_index in kf.split(train_tfidf): # Data split
    X_train, X_test = train_tfidf[train_index], train_tfidf[test_index]
    y_train, y_test = y[train_index], y[test_index]
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    metrics = model.evaluate(predictions, y_test)
    accuracies.append(metrics['accuracy'])
    fold += 1
print(f'Average Accuracy: {np.mean(accuracies):.2f}')
end_time = time.time()
elapsed_time = end_time - start_time
print(f" Training & prediction time: {elapsed_time:.2f} seconds")

```

```

model = BernoulliNaiveBayes()
accuracies = []
fold = 1
start_time = time.time()
for train_index, test_index in kf.split(X_train_combined_dense):
    X_train, X_test = X_train_combined_dense[train_index], X_train_combined_dense[test_index]
    y_train, y_test = y[train_index], y[test_index]
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    metrics = model.evaluate(predictions, y_test)
    accuracies.append(metrics['accuracy'])
    fold += 1
print(f'Average Accuracy: {np.mean(accuracies):.2f}')
end_time = time.time()
elapsed_time = end_time - start_time
print(f" Training & prediction time: {elapsed_time:.2f} seconds")

```

```

#Basic training data
model = BernoulliNaiveBayes()
accuracies = []
fold = 1
start_time = time.time()
for train_index, test_index in kf.split(X_train_basic):
    X_train, X_test = X_train_basic[train_index], X_train_basic[test_index]
    y_train, y_test = y[train_index], y[test_index]
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    metrics = model.evaluate(predictions, y_test)
    accuracies.append(metrics['accuracy'])
    fold += 1
print(f'average accuracy: {np.mean(accuracies):.2f}')
end_time = time.time()
elapsed_time = end_time - start_time
print(f" Training and Prediction Time: {elapsed_time:.2f} seconds")

```

```

dataset = ("Basic", X_train_basic, "TF-IDF", train_tfidf, "Final", X_train_combined_dense)
t = 0
model = [
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "SVC": SVC(kernel="linear", C=0.1),
    "BernoulliNB": BernoulliNB(),
    "MLPClassifier": MLPClassifier(),
    "GradientBoostingClassifier": GradientBoostingClassifier()
]
kf = KFold(n_splits=10, shuffle=True, random_state=42)
result_df = []
for dataset_name, dataset_value in dataset.items():
    for model_name, model_value in model.items():
        accuracies = []
        start_time = time.time()
        for train_index, test_index in kf.split(dataset_value):
            X_train, X_test = dataset_value[train_index], dataset_value[test_index]
            y_train, y_test = t[train_index], t[test_index]
            model_value.fit(X_train, y_train)
            predictions = model_value.predict(X_test)
            accuracy = accuracy_score(y_test, predictions)
            accuracies.append(accuracy)
        end_time = time.time()
        elapsed_time = end_time - start_time
        result_df.append([dataset_name, model_name, "average accuracy", np.mean(accuracies), "prediction and training time", elapsed_time])
        print(f"Completed {model_name} with {dataset_name}")
result_df = pd.DataFrame(result_df)
print(result_df)
result_df.to_csv('results.csv', index=False)

```

Other classifiers

```
] # At least one more classifier like Logistic Regression, Decision Tree

from sklearn.linear_model import LogisticRegression
model_lr = LogisticRegression()

from sklearn.tree import DecisionTreeClassifier
model_dt = DecisionTreeClassifier()

from sklearn.ensemble import RandomForestClassifier
model_rf = RandomForestClassifier(n_estimators=100, random_state=42)

from sklearn.svm import SVC
model_svm = SVC(kernel='linear', C=1.0)

from sklearn.neighbors import KNeighborsClassifier
model_knn_1 = KNeighborsClassifier(n_neighbors=1)
model_knn_3 = KNeighborsClassifier(n_neighbors=3)
model_knn_5 = KNeighborsClassifier(n_neighbors=5)
model_knn_7 = KNeighborsClassifier(n_neighbors=7)
model_knn_41 = KNeighborsClassifier(n_neighbors=41)
model_knn_11 = KNeighborsClassifier(n_neighbors=11)
model_knn_15 = KNeighborsClassifier(n_neighbors=15)
model_knn_21 = KNeighborsClassifier(n_neighbors=21)
model_knn_33 = KNeighborsClassifier(n_neighbors=33)

from xgboost import XGBClassifier
model_xgb = XGBClassifier(use_label_encoder=False, eval_metric='mlogloss')
```

Voting Classifier

```
] processed_X_train = pd.read_csv('processed_X_train.csv', header=None)
processed_X_train = np.array(processed_X_train)
processed_X_test = pd.read_csv('processed_X_test.csv', header=None)
processed_X_test = np.array(processed_X_test)

train_tfidf = pd.read_csv('train_tfidf.csv', header=None)
train_tfidf = np.array(train_tfidf)
test_tfidf = pd.read_csv('test_tfidf.csv', header=None)
test_tfidf = np.array(test_tfidf)

X_train_basic = pd.read_csv('basic_train_features.csv', header=None)
X_train_basic = np.array(X_train_basic)
X_test_basic = pd.read_csv('basic_test_features.csv', header=None)
X_test_basic = np.array(X_test_basic)

y_labels = pd.read_csv('train_labels.csv', header=None)
y_labels = y_labels.values.ravel()
```

```
from sklearn.ensemble import VotingClassifier
voting=[]

nb = BernoulliNB(alpha = 0.1)
lr = LogisticRegression()
dt = DecisionTreeClassifier()
rf = RandomForestClassifier(n_estimators=100, random_state=42)
mlp = MLPClassifier()

voting_clf = VotingClassifier(estimators=[('nb', nb), ('lr', lr), ('dt', dt), ('rf', rf), ('mlp', mlp)], voting='soft')
accuracies = []
kf = KFold(n_splits=10, shuffle=True, random_state=42)
start_time = time.time()
for train_index, test_index in kf.split(processed_X_train):
    X_train, X_test = processed_X_train[train_index], processed_X_train[test_index]
    y_train, y_test = y_labels[train_index], y_labels[test_index]
    voting_clf.fit(X_train, y_train)
    predictions = voting_clf.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)
    accuracies.append(accuracy)
print("Accuracy: (accuracy:.2f)")

end_time = time.time()
elapsed_time = end_time - start_time
print("Average Accuracy: (np.mean(accuracies):.2f)")
print("Training and Prediction Time: (elapsed_time:.2f) seconds")

voting.append(("Dataset": "Combined", "Voting": "soft", "average accuracy": np.mean(accuracies), "prediction and training time": elapsed_time))
```

```
from sklearn.ensemble import VotingClassifier
voting=[]

nb = BernoulliNB(alpha = 0.1)
lr = LogisticRegression()
dt = DecisionTreeClassifier()
rf = RandomForestClassifier(n_estimators=100, random_state=42)
mlp = MLPClassifier()

voting_clf = VotingClassifier(estimators=[('nb', nb), ('lr', lr), ('dt', dt), ('rf', rf), ('mlp', mlp)], voting='hard')
accuracies = []
kf = KFold(n_splits=10, shuffle=True, random_state=42)
start_time = time.time()
for train_index, test_index in kf.split(processed_X_train):
    X_train, X_test = processed_X_train[train_index], processed_X_train[test_index]
    y_train, y_test = y_labels[train_index], y_labels[test_index]
    voting_clf.fit(X_train, y_train)
    predictions = voting_clf.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)
    accuracies.append(accuracy)
print("Accuracy: (accuracy:.2f)")

end_time = time.time()
elapsed_time = end_time - start_time
print("Average Accuracy: (np.mean(accuracies):.2f)")
print("Training and Prediction Time: (elapsed_time:.2f) seconds")

voting.append(("Dataset": "Combined", "Voting": "hard", "average accuracy": np.mean(accuracies), "prediction and training time": elapsed_time))
```

```
from sklearn.ensemble import VotingClassifier
voting=[]

nb = BernoulliNB(alpha = 0.1)
lr = LogisticRegression()
dt = DecisionTreeClassifier()
rf = RandomForestClassifier(n_estimators=100, random_state=42)
mlp = MLPClassifier()

voting_clf = VotingClassifier(estimators=[('nb', nb), ('lr', lr), ('dt', dt), ('rf', rf), ('mlp', mlp)], voting='soft')
accuracies = []
kf = KFold(n_splits=10, shuffle=True, random_state=42)
start_time = time.time()
for train_index, test_index in kf.split(train_tfidf):
    X_train, X_test = train_tfidf[train_index], train_tfidf[test_index]
    y_train, y_test = y_labels[train_index], y_labels[test_index]
    voting_clf.fit(X_train, y_train)
    predictions = voting_clf.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)
    accuracies.append(accuracy)
print("Accuracy: (accuracy:.2f)")

end_time = time.time()
elapsed_time = end_time - start_time
print("Average Accuracy: (np.mean(accuracies):.2f)")
print("Training and Prediction Time: (elapsed_time:.2f) seconds")

voting.append(("Dataset": "Combined", "Voting": "soft", "average accuracy": np.mean(accuracies), "prediction and training time": elapsed_time))
```



```

# More Model Combinations
model_combinations = []

# Two-model combinations
"nb" ["nb", BernoulliNB(alpha=0.1), "mlp", MLPClassifier(hidden_layer_sizes=(50,10)),
      "log" ["log", LogisticRegression(), "gb", GradientBoostingClassifier(estimators=10), learning_rate=0.01],

# Three-model combinations
"rfp" ["rfp", BernoulliNB(alpha=0.1), "svc", SVC(kernel="linear", C=0.1, probability=True), "gbr", RandomForestClassifier(estimators=100, learning_rate=0.01),
      "dtr" ["dtr", DecisionTreeClassifier(max_depth=10, "mlp", MLPClassifier(hidden_layer_sizes=(50,10,10))),
      "lgb" ["lgb", BernoulliNB(alpha=0.1), "l", LogisticRegression(), "gb", GradientBoostingClassifier(estimators=100, learning_rate=0.01),
      "xgb" ["xgb", XGBClassifier(estimators=100, learning_rate=0.01), "gbr", RandomForestClassifier(estimators=100, learning_rate=0.01), "mlp", MLPClassifier(hidden_layer_sizes=(50,10,10)),
      "lgb" ["lgb", BernoulliNB(alpha=0.1), "l", LogisticRegression(), "gb", GradientBoostingClassifier(estimators=100, learning_rate=0.01),

# Four-model combinations
"rfp" ["rfp", BernoulliNB(alpha=0.1), "l", LogisticRegression(), "l", RandomForestClassifier(estimators=100),
      "gbr", GradientBoostingClassifier(estimators=100, learning_rate=0.01), "gbr", GradientBoostingClassifier(estimators=100, learning_rate=0.01),
      "mlp", MLPClassifier(hidden_layer_sizes=(50,10,10)), "l", RandomForestClassifier(estimators=100),
      "dtr" ["dtr", BernoulliNB(alpha=0.1), "l", LogisticRegression(), "l", RandomForestClassifier(estimators=100),
      "lgb" ["lgb", BernoulliNB(alpha=0.1), "l", LogisticRegression(), "l", RandomForestClassifier(estimators=100),
      "xgb" ["xgb", XGBClassifier(estimators=100, learning_rate=0.01), "l", RandomForestClassifier(estimators=100, learning_rate=0.01),
      "lgb" ["lgb", XGBClassifier(estimators=100, learning_rate=0.01), "l", RandomForestClassifier(estimators=100, learning_rate=0.01),
      "xgb", XGBClassifier(), "dtr", probability=True], "mlp", MLPClassifier(hidden_layer_sizes=(50,10,10)),

# Six-model combinations
"rfp" ["rfp", BernoulliNB(alpha=0.1), "l", LogisticRegression(), "l", RandomForestClassifier(estimators=100),
      "gbr", GradientBoostingClassifier(estimators=100, learning_rate=0.01), "gbr", XGBClassifier(estimators=100, learning_rate=0.01),
      "mlp", MLPClassifier(hidden_layer_sizes=(50,10,10)), "l", RandomForestClassifier(estimators=100),
      "dtr" ["dtr", BernoulliNB(alpha=0.1), "l", LogisticRegression(), "l", RandomForestClassifier(estimators=100),
      "lgb" ["lgb", BernoulliNB(alpha=0.1), "l", LogisticRegression(), "l", RandomForestClassifier(estimators=100),
      "xgb" ["xgb", XGBClassifier(estimators=100, learning_rate=0.01), "l", RandomForestClassifier(estimators=100, learning_rate=0.01),
      "lgb" ["lgb", XGBClassifier(estimators=100, learning_rate=0.01), "l", RandomForestClassifier(estimators=100, learning_rate=0.01),
      "xgb", XGBClassifier(), "dtr", probability=True], "mlp", MLPClassifier(hidden_layer_sizes=(50,10,10))
]

```

```

# I removed hard, because hard seems to generally lower than soft
voting_type = "soft"

X_train = pd.read_csv('processed_X_train.csv', header=None).values
y_train = pd.read_csv('train_labels.csv', header=None).values.ravel()

kf = KFold(n_splits=10, shuffle=True, random_state=2)
results = []

for voting_type in voting_types:
    for key, models in model_combinations.items():
        print(f'Evaluating {key} with {voting_type} voting...')

        voting_clf = VotingClassifier(estimators=models, voting=voting_type)
        accuracies = []

        start_time = time.time()

        for train_index, test_index in kf.split(X_train):
            X_tr, X_te = X_train[train_index], X_train[test_index]
            y_tr, y_te = y_train[train_index], y_train[test_index]

            voting_clf.fit(X_tr, y_tr)
            predictions = voting_clf.predict(X_te)
            accuracy = accuracy_score(y_te, predictions)
            accuracies.append(accuracy)

        end_time = time.time()
        elapsed_time = end_time - start_time
        avg_acc = np.mean(accuracies)

        results.append(("Model Set": key, "Voting Type": voting_type, "Average Accuracy": avg_acc, "Time Taken (s)": elapsed_time))
        print(f'{key} ({voting_type}) - Accuracy: {avg_acc:4f}, Time: {elapsed_time:2f}s')

```

```

# Model Combinations with Default Parameters
model_combinations = []

# Two-model combinations
"nb" ["nb", BernoulliNB(), "mlp", MLPClassifier()], # Default
"log" ["log", LogisticRegression(), "gb", GradientBoostingClassifier()], # Default

# More Three-model combinations
"rfp" ["rfp", BernoulliNB(), "l", LogisticRegression(), "gbr", RandomForestClassifier()], # Default
"dtr" ["dtr", DecisionTreeClassifier(), "l", RandomForestClassifier()], "mlp", MLPClassifier()], # Default
"lgb" ["lgb", BernoulliNB(), "l", LogisticRegression(), "xgb", XGBClassifier()], # Default
"xgb" ["xgb", XGBClassifier(), "l", RandomForestClassifier()], "gbr", MLPClassifier()], # Default
"lgb" ["lgb", BernoulliNB(), "l", LogisticRegression(), "gbr", GradientBoostingClassifier()], # Default

# More Four-model combinations
"rfp" ["rfp", BernoulliNB(), "l", LogisticRegression(), "l", RandomForestClassifier()], "gbr", GradientBoostingClassifier()], # Default
"rfp" ["rfp", SVC(probability=True), "gbr", XGBClassifier()], "gbr", MLPClassifier()], "gbr", RandomForestClassifier()], # Default
"rfp" ["rfp", BernoulliNB(), "l", LogisticRegression(), "l", DecisionTreeClassifier()], "gbr", GradientBoostingClassifier()], # Default
"rfp" ["rfp", XGBClassifier(), "l", RandomForestClassifier()], "xgb", XGBClassifier()], "mlp", MLPClassifier()], # Default

# Six-model combinations
"rfp" ["rfp", BernoulliNB(), "l", LogisticRegression(), "l", RandomForestClassifier()],
      "gbr", GradientBoostingClassifier()], "gbr", XGBClassifier()], "xgb", SVC(probability=True)], # Default
"rfp" ["rfp", BernoulliNB(), "l", LogisticRegression(), "l", RandomForestClassifier()],
      "gbr", GradientBoostingClassifier()], "l", LogisticRegression(), "l", MLPClassifier()], # Default

# Seven-model combinations
"rfp" ["rfp", BernoulliNB(), "l", LogisticRegression(), "l", RandomForestClassifier()],
      "gbr", GradientBoostingClassifier()], "gbr", XGBClassifier()], "xgb", SVC(probability=True)], # Default
"rfp" ["rfp", BernoulliNB(), "l", LogisticRegression(), "l", RandomForestClassifier()],
      "gbr", GradientBoostingClassifier()], "l", LogisticRegression(), "l", MLPClassifier()], # Default

```

```

kf = KFold(n_splits=10, shuffle=True, random_state=2)
results = []

# Run different model combinations
for voting_type in voting_types:
    for key, models in model_combinations.items():
        print(f'Evaluating {key} with {voting_type} voting...')

        voting_clf = VotingClassifier(estimators=models, voting=voting_type)
        accuracies = []

        start_time = time.time()

        for train_index, test_index in kf.split(X_train):
            X_tr, X_te = X_train[train_index], X_train[test_index]
            y_tr, y_te = y_train[train_index], y_train[test_index]

            voting_clf.fit(X_tr, y_tr)
            predictions = voting_clf.predict(X_te)
            accuracy = accuracy_score(y_te, predictions)
            accuracies.append(accuracy)

        end_time = time.time()
        elapsed_time = end_time - start_time
        avg_acc = np.mean(accuracies)

        results.append(("Model Set": key, "Voting Type": voting_type, "Average Accuracy": avg_acc, "Time Taken (s)": elapsed_time))
        print(f'{key} ({voting_type}) - Accuracy: {avg_acc:4f}, Time: {elapsed_time:2f}s')

```

Best model

```

} from sklearn.ensemble import VotingClassifier

nb = BernoulliNB(alpha = 0.1)
lr = LogisticRegression()
dt = DecisionTreeClassifier()
rf = RandomForestClassifier(n_estimators=100, random_state=2)
mlp = MLPClassifier()

model_test = VotingClassifier(estimators=[('nb', nb), ('lr', lr), ('dt', dt), ('rf', rf), ('mlp', mlp)], voting='hard')

model_test.fit(X_train_basic, y_labels)
predictions = model_test.predict(X_test_basic)

output = pd.DataFrame({'subreddit': predictions})
output.index.name = 'id'
print(output.head())

output.to_csv('results.csv')

```